

Local Backend Architecture Guide for Electron + Blazor + .NET

Local Backend Architecture Guide for Electron + Blazor + .NET

This document provides a complete, end-to-end explanation of the architecture we established: a **local-only backend running in its own process**, an **Electron shell**, and a **Blazor frontend** communicating through a clean, C#-friendly API. It describes the roles of each layer, the communication flow, and the recommended implementation patterns.

1. Architectural Overview

The system consists of three independent layers:

1. Backend Process (.NET)

- Runs as a completely separate OS process.
- Never embedded as a DLL.
- Never remote.
- Communicates via **stdio** or **named pipes**.
- Implements orchestration, logic, workflows, and agent operations.

2. Electron Main Process (Node.js)

- Acts as the **bridge** between frontend and backend.
- Spawns the backend process.
- Manages IPC between renderer and backend.
- Handles OS integration, window management, and menus.

3. Frontend (Blazor WebAssembly)

- Runs inside Electron's renderer process.
- Provides the UI.
- Communicates with Electron main through a **single C# service** using JS interop.
- Never interacts directly with stdio, pipes, or Node APIs.

This separation ensures a clean, explicit, acyclic architecture.

2. Communication Flow

The communication chain is strictly linear and controlled:

```
Blazor (C#)
↓ JS Interop
Electron Preload (window.backendBridge)
↓ Electron IPC
Electron Main Process
↓ stdio / pipes
Backend Process (.NET)
```

Key principles:

- The **frontend never talks directly** to the backend.
 - The **main process is the only component** allowed to spawn and communicate with the backend.
 - The frontend uses a **C# service** that hides all JavaScript details.
 - The backend uses **stdin/stdout** or **pipes** for message exchange.
-

3. Responsibilities of Each Layer

Backend (.NET)

- Implements all business logic.
- Reads messages from stdin.
- Writes responses to stdout.
- Uses JSON or a custom binary protocol.
- Remains fully local and isolated.

Electron Main Process

- Spawns the backend using `child_process.spawn()`.
- Listens to backend stdout.
- Writes to backend stdin.
- Forwards messages to the renderer via IPC.
- Receives renderer requests and forwards them to the backend.

Frontend (Blazor)

- Renders UI.

- Calls a single C# service to send messages.
 - Receives backend responses through JS events.
 - Avoids direct JS usage except through a minimal interop layer.
-

4. Preload Layer: Exposing a Safe API

Electron's preload script exposes a minimal, stable API to the frontend:

- `backendBridge.send(message)` - sends a message to the backend.
- `backendBridge.onMessage(callback)` - receives messages from the backend.

This API is the only JavaScript surface the frontend interacts with.

5. Blazor Integration Through a C# Service

Blazor interacts with Electron through a **single injected service**:

- Provides a clean C# method: `SendAsync(object message)`.
- Handles serialization.
- Subscribes to backend events.
- Exposes a C# event for backend messages.

This ensures the frontend remains C#-centric and avoids JS complexity.

6. Message Protocol

Messages between layers follow a simple, consistent structure:

- JSON objects with a `cmd` field.
- Optional payload fields.
- Responses follow the same pattern.

Example:

```
{  
  "cmd": "ping"  
}
```

Backend response:

```
{  
  "result": "pong"  
}
```

This protocol is easy to extend and version.

7. Benefits of This Architecture

Performance

- No HTTP, REST, or gRPC overhead.
- Local stdio/pipes are extremely fast.

Isolation

- Backend runs independently.
- Crashes do not affect the UI.

Simplicity

- No servers.
- No networking.
- No DLL loading.
- No remote deployment.

Clean Separation

- UI, orchestration, and shell responsibilities are clearly divided.
- No circular dependencies.
- No hidden recursion.

Blazor-Friendly

- Only one JS interop entry point.
 - All logic stays in C#.
 - Electron complexity is hidden.
-

8. Recommended Folder Structure

```
project-root/  
  electron/  
  main.js  
  preload.js  
  frontend/  
  blazor-app/  
  Pages/  
  Shared/  
  Services/BackendService.cs  
  backend/  
  MyBackend/  
  Program.cs
```

This structure keeps each layer independent and testable.

9. Summary

This architecture provides a robust, explicit, and maintainable foundation for a desktop application where:

- The backend is always local.
- The backend always runs in its own process.
- Electron acts as a shell and IPC bridge.
- Blazor provides the UI.
- Communication uses stdio or pipes.
- The frontend interacts through a clean C# service.

It avoids unnecessary complexity such as REST, HTTP, gRPC, or DLL loading, while ensuring strong separation of concerns and a predictable workflow.

10. Next Steps

Potential enhancements include:

- Defining a typed message schema.
- Adding structured logging for backend communication.
- Implementing a supervisor in Electron to restart the backend if it crashes.
- Introducing versioned commands for long-term stability.

This guide serves as the foundation for building a clean, explicit, and high-performance local desktop application architecture using Electron, Blazor, and .NET.